Metadata handling for Open Access Journal PDFs

Cite as: Martin Paul Eve, "Metadata handling for Open Access Journal PDFs", *eve.gd: Martin Paul Eve*, June 06, 2012, https://doi.org/10.59348/2gt43-gq719.

As I count down to the launch of Orbit: Writing around Pynchon, I've been thinking carefully about the mechanisms through which the articles will be consumed. In short: what metadata should be in the PDFs and where should it be.

Obviously, I want the metadata to be visible to the human eye, but what about embedding this within the PDF's proper metadata mechanism? Apache FOP, which I'm using to the transforms, has the facility to do this. However, do other journals bother?

Here's a metadata dump using pdftk on a top-rank Taylor and Francis journal in English literature:

> *InfoKey: Producer*
> *InfoValue: iText 2.1.4 (by lowagie.com)*
> *InfoKey: ModDate*
> *InfoValue: D:20101227134204Z*
> *InfoKey: CreationDate*
> *InfoValue: D:20101227134204Z*
> *PdfID0: da625abeee725c7372c85bab42a58ff9*
> *PdfID1: a738f6173b5722dbf66507c0289aa1*
> *NumberOfPages: 17*

That's not especially descriptive!

By contrast, my XSL transform is producing the following:

InfoKey: Creator

InfoValue: meXml: Martin Eve's XML Generator. https://www.martineve.com/

InfoKey: Title

InfoValue: Generating PDFs from OJS

InfoKey: Producer

InfoValue: Apache FOP Version 1.0

InfoKey: Author

InfoValue: Martin Paul Eve

InfoKey: Subject

InfoValue: It has long been desirable to create PDF files from a standard XML base. This plugin allows that to happen using a combination of OJS, Saxon and FOP.

InfoKey: CreationDate

InfoValue: D:20120605175241+01'00'

PdfID0: f2e62132fce56dea2a80dccf6703b95

PdfID1: f2e62132fce56dea2a80dccf6703b95

NumberOfPages: 4

PageLabelNewIndex: 1

PageLabelStart: 1

PageLabelPrefix: 1

PageLabelNumStyle: NoNumber

PageLabelNewIndex: 2

PageLabelStart: 1

PageLabelPrefix: 1

PageLabelNumStyle: NoNumber

PageLabelNewIndex: 3

PageLabelStart: 1

PageLabelPrefix: 2

PageLabelNumStyle: NoNumber

PageLabelNewIndex: 4

> *PageLabelStart: 1*
>
> *PageLabelPrefix: 3*
>
> *PageLabelNumStyle: NoNumber*

However, interestingly, the Taylor and Francis journal can be perfectly detected by Zotero. So where is it getting its info?

The great JISC document on PDF metadata extraction mechanisms has the following for Zotero:

> *Zotero uses "Google Scholar Results as well as DOIs on the first page to get metadata and that works in a large majority of cases". This implies that metadata extraction relies on converting the PDF to text at the client, using Regular Expressions to detect the DOI string, and submitting that string to Google Scholar or doi.org to retrieve the matching record.*

All sounds good. So, as a test, I changed the DOI in my test document to reflect an article that I know worked. I changed the author, Title and DOI to all match the second article. I even put in a URL pointer to dx.doi.org/.....

However, Zotero still wouldn't pick it up; it completely mis-identifies it. So I decided to dive into the mechanics.

Zotero's main PDF functionality resides in recognizePDF.js. Here's the first part of that function:

```
const MAX_PAGES = 3;

const lineRe = /^\s*([^\s]+(?: [^\s]+)+)/;

this._libraryID = libraryID;
this._callback = callback;
//this._captchaCallback = captchaCallback;

var cacheFile = Zotero.getZoteroDirectory();
cacheFile.append("recognizePDFcache.txt");
if(cacheFile.exists()) {
cacheFile.remove(false);
}

Zotero.debug('Running pdftotext -enc UTF-8 -nopgbrk '
+ '-l ' + MAX_PAGES + ' "' + file.path + '" "'
+ cacheFile.path + '"');

var proc = Components.classes["@mozilla.org/process/util;1"].
createInstance(Components.interfaces.nsIProcess);
var exec = Zotero.getZoteroDirectory();
exec.append(Zotero.Fulltext.pdfConverterFileName);
proc.init(exec);

var args = ['-enc', 'UTF-8', '-nopgbrk', '-layout', '-l', MAX_PAGES];
args.push(file.path, cacheFi10.1080/095023608022637821e.path);
try {
if (!Zotero.isFx36) {
proc.runw(true, args, args.length);
}
else {
proc.run(true, args, args.length);
}
}
catch (e) {
Zotero.debug("Error running pdfinfo", 1);
Zotero.debug(e, 1);
}

if(!cacheFile.exists()) {
this._callback(false, "recognizePDF.couldNotRead");
return;
}

var inputStream = Components.classes["@mozilla.org/network/file-input-stream;1"]
.createInstance(Components.interfaces.nsIFileInputStream);
inputStream.init(cacheFile, 0x01, 0664, 0);
```

```
var intlStream = Components.classes["@mozilla.org/intl/converter-input-stream;1"]
.createInstance(Components.interfaces.nsIConverterInputStream);
intlStream.init(inputStream, "UTF-8", 65535,
Components.interfaces.nsIConverterInputStream.DEFAULT_REPLACEMENT_CHARACTER);
intlStream.QueryInterface(Components.interfaces.nsIUnicharLineInputStream);

// get the lines in this sample
var lines = [];
var lineLengths = [];
var str = {};
while(intlStream.readLine(str)) {
var line = lineRe.exec(str.value);
if(line) {
lines.push(line[1]);
lineLengths.push(line[1].length);
}
}

inputStream.close();
cacheFile.remove(false);
```

This first code block runs pdftotext on the file. The command it assembles looks somewhat like this: pdftotext -enc UTF-8 -nopgbrk -l '3' new.pdf /your/zotero/directory/recognizePDFcache.txt.

So far so good. The output I got looked a little like this:

> *Orbit: Writing Around Pynchon*
>
> *https://www.pynchon.net*
>
> *ISSN: 2044-4095*
>
> *Author(s):*
>
> *Affiliation(s):*
>
> *Title:*
>
> *Date:*
>
> *Volume:*
>
> *Issue:*
>
> *URL:*
>
> *DOI:*
>
> *Author Name Redacted*
>
> *University of Sussex*
>
> *Title Redacted*
>
> *28 September 2011*
>
> *1*
>
> *1*
>
> *http://dx.doi.org/10.___/_____*
>
> *10.___/_____*
>
> *Abstract:*
>
> *It has long been desirable to create PDF files from a standard XML base.*

The remainder of this function makes an informed guess as to which type of document it's dealing with.

```
// look for DOI
var allText = lines.join("\n");
Zotero.debug(allText);
var m = Zotero.Utilities.cleanDOI(allText);
if(m) {
this._DOI = m[0];
}

// get (not quite) median length
var lineLengthsLength = lineLengths.length;
if(lineLengthsLength < 20
|| lines[0] === "This is a digital copy of a book that was preserved for generations on
library shelves before it was carefully scanned by Google as part of a project") {
this._callback(false, "recognizePDF.noOCR");
} else {
var sortedLengths = lineLengths.sort();
var medianLength = sortedLengths[Math.floor(lineLengthsLength/2)];

// pick lines within 4 chars of the median (this is completely arbitrary)
this._goodLines = [];
var uBound = medianLength + 4;
var lBound = medianLength - 4;
for (var i=0; i<lineLengthsLength; i++) {
if(lineLengths[i] > lBound && lineLengths[i] < uBound) {
// Strip quotation marks so they don't mess up search query quoting
var line = lines[i].replace('"', '');
this._goodLines.push(line);
}
}

this._startLine = this._iteration = 0;
this._queryGoogle();
}
```

First off, it amalgamates the lines and passes them to the cleanDOI function. This performs a string.match:

```
x.match(/10\.[0-9]{4,}\/[^\s]*[^\s\.,]/)
```

I can confirm that my DOI passes the match test here.

The next step Zotero takes is to work out how many lines are in the document. If there are fewer than 20 lines, it assumes that the document doesn't contain OCRed text and returns a fail.

As you can see, though, Zotero also has a debug function, so I enabled that at this point. When I looked in the log, the DOI number was not being picked up by Zotero's internal pdftotext. In fact, Zotero's version of pdfttotext seems to disregard anything inside a block!</p>

The second I put the DOI number in a non-table area, it was detected.

tl;dr: make sure your DOI numbers are somewhere that Zotero's version of pdftotext can read it.

Featured image by TJOwens under a CC-BY license.